

Lab4 实验报告

姓名：朱文凯

学号：23307110192

1. 实验目标

本次 Lab4 的目标是在已有五级流水线 CPU 上加入 CSR (Control and Status Register) 相关支持，并通过 `make test-lab4` 测试。

根据实验要求，本次需要实现 6 条 CSR 指令：

- CSRRW
- CSRRS
- CSRRC
- CSRRWI
- CSRRSI
- CSRRCI

需要支持的必做 CSR 包括

`mstatus`、`mtvec`、`mip`、`mie`、`mscratch`、`mcause`、`mtval`、`mepc`、`mcycle`、`mhartid` 和 `satp`。此外，我也实现了 `csr.sv` 中给出的 S 模式相关 CSR，包括 `sstatus`、`stvec`、`sscratch`、`sepc`、`scause`、`stval`、`sie` 和 `sip`，并补充了 `medeleg`、`mideleg`、`pmpcfg0`、`pmpaddr0` 的基本读写支持。

本次实验的重点不只是增加几条指令，还包括：

- CSR 读写语义是否和 RISC-V 规则一致
- CSR 写 `mask` 是否正确处理
- `mcycle` 是否每周期自增并支持写入覆盖
- `sstatus` 是否作为 `mstatus` 的子集处理
- CSR 指令后是否正确刷新流水线
- `Diffptest` 中的 CSR 状态是否和 CPU 内部状态一致

2. 总体设计思路

2.1 增加独立的 CSR 控制语义

我没有把 CSR 指令伪装成普通 ALU 指令，而是在译码结果中单独加入 CSR 控制字段。

主要新增字段包括：

- `is_csr`：当前指令是否为 CSR 指令
- `csr_op`：表示写、置位或清位
- `csr_addr`：CSR 地址
- `csr_uses_imm`：是否使用 5 位立即数 `zimm`
- `csr_zimm`：零扩展后的 CSR 立即数

这样后续流水线可以明确区分普通寄存器写回、访存写回、`pc + 4` 写回和 CSR 旧值写回，避免把 CSR 行为混进 ALU 控制里。

2.2 用独立 CSR 模块集中维护状态

本次新增了 `core_csr.sv`，集中负责：

- CSR 读
- CSR 写
- `CSRRW` / `CSRRS` / `CSRRC` 的写入值计算
- 各 CSR 的写 mask
- `mcycle` 每周期自增
- `mhartid` 恒为 0
- `Difftest` 需要的 CSR 状态导出

这样做的好处是 `core.sv` 只需要处理流水线数据通路和提交时机，CSR 自身的寄存器语义集中在一个模块里，后续扩展异常、中断或 `mret` 时也更容易维护。

2.3 CSR 写入在 WB 阶段提交

CSR 指令在 EX 阶段读取 CSR 旧值，用它作为 `rd` 的写回数据；同时根据 `rs1` 或 `zimm` 计算 CSR 写源。

真正的 CSR 写入放在 WB 阶段，并且只在 `commit_valid_wb` 成立时发生。这样可以 Let CSR 状态变化、GPR 写回和 `Difftest` 提交保持同一个提交顺序。

对于 `CSRRS` / `CSRRC`：

- 如果是寄存器版本且 $rs1 = x0$ ，只读 CSR，不写 CSR
- 如果是立即数版本且 $zimm = 0$ ，只读 CSR，不写 CSR

CSRRW / CSRRWI 则正常执行写入。

2.4 CSR 指令后刷新流水线

实验要求中说明 CSR 不应该转发，每次 CSR 改变后都需要刷新流水线。当前 CPU 已经有分支和跳转的重定向逻辑，因此我复用了这条路径：

- CSR 指令到 EX 阶段后产生一次重定向
- 重定向目标为 $pc_{ex} + 4$
- 清空 IF/ID 和 ID/EX 中的年轻指令

这种策略比较保守，但能保证 CSR 后面的指令重新从正确状态下取指，不依赖 CSR forwarding。

3. 主要实现内容

3.1 common.sv

在公共类型中增加：

- WB_CSR 写回来源
- csr_op_t
 - CSR_OP_WRITE
 - CSR_OP_SET
 - CSR_OP_CLEAR
- decode_out_t 中的 CSR 控制字段

这样 CSR 指令可以沿着现有五级流水线传递控制信息。

3.2 core_decode.sv

在 SYSTEM opcode (7'b11110011) 下新增 CSR 指令译码。

译码规则如下：

- funct3 = 001 : CSRRW
- funct3 = 010 : CSRRS

- funct3 = 011 : CSRRRC
- funct3 = 101 : CSRRWI
- funct3 = 110 : CSRRSI
- funct3 = 111 : CSRRCI

CSR 指令的写回来源设为 `WB_CSR`，写回数据是 CSR 的旧值。

3.3 core_csr.sv

`core_csr.sv` 是本次新增的核心模块。

它实现了实验要求的必做 CSR：

- `mstatus`
- `mtvec`
- `mip`
- `mie`
- `mscratch`
- `mcause`
- `mtval`
- `mepc`
- `mcycle`
- `mhartid`
- `satp`

同时也实现了 `bonus` 中的 S 模式相关 CSR：

- `sstatus`
- `stvec`
- `sscratch`
- `sepc`
- `scause`
- `stval`
- `sie`
- `sip`

其中 `sstatus` 不单独保存，而是由 `mstatus & SSTATUS_MASK` 得到。写 `sstatus` 时，只更新 `mstatus` 中对应的可写位。

对于写 `mask`：

- mstatus 使用 MSTATUS_MASK
- mtvec 使用 MTVEC_MASK
- mip 使用 MIP_MASK
- medeleg 使用 MEDELEG_MASK
- mideleg 使用 MIDELEG_MASK

mcycle 每个周期自动加一，如果 CSR 指令写入 mcycle，则使用写入值覆盖自动加一结果。mhartid 固定为 0，写入会被忽略。

3.4 core.sv

core.sv 中主要做了三类修改。

第一类是流水线寄存器扩展：

将 CSR 地址、操作类型、立即数、读值和写源从 ID/EX 一直传到 MEM/WB。

第二类是 CSR 写回路径：

在 WB 阶段，WB_CSR 会把 CSR 旧值写回到 rd。CSR 自身的写入也在 WB 阶段随 commit_valid_wb 一起发生。

第三类是流水线刷新：

当 EX 阶段出现有效 CSR 指令时，重定向到 pc_ex + 4，并清空年轻流水线级。

3.5 Difftest 连接

本次把原先接 0 的 DifftestCSRState 改为连接真实 CSR 状态：

- mstatus
- sstatus
- mepc
- sepc
- mtval
- stval
- mtvec
- stvec
- mcause
- scause
- satp
- mip
- mie

- mscratch
- sscratch
- mideleg
- medeleg

同时将 `DifftestInstrCommit`、`DifftestArchIntRegState`、`DifftestTrapEvent` 和 `DifftestCSRState` 的 `coreid` 改为 `mhartid[7:0]`。

调试时发现 CSR 状态也需要像 GPR 一样对 `Difftest` 做提交旁路。否则第一条 `csrw mstatus` 提交时，`Difftest` 会看到旧的 `mstatus`，产生晚一拍的 `mismatch`。最终在 CSR 模块的 `Difftest` 输出侧加入了提交视图，使提交当拍能看到写入后的 CSR 状态。

4. CSR 寄存器作用简述

本次实验中涉及的主要 CSR 作用如下：

- `mstatus`：机器模式状态寄存器，保存全局中断使能、特权级保存等状态位。
- `sstatus`：`mstatus` 中和 S 模式相关的字段视图。
- `mtvec` / `stvec`：trap 入口地址寄存器。
- `mip` / `sip`：中断 pending 状态寄存器。
- `mie` / `sie`：中断使能寄存器。
- `mscratch` / `sscratch`：异常处理时可使用的临时寄存器。
- `mepc` / `sepc`：发生异常或中断时保存的返回 PC。
- `mcause` / `scause`：记录 trap 原因。
- `mtval` / `stval`：记录 trap 附加信息，例如出错地址或非法指令信息。
- `mcycle`：记录 CPU 已运行的时钟周期数。
- `mhartid`：记录当前硬件线程编号。本实验是单核 CPU，因此恒为 0。
- `satp`：地址转换与保护相关寄存器，后续虚拟内存相关实验会用到。
- `medeleg` / `mideleg`：异常和中断委托寄存器，本实验中按给定 `mask` 实现。
- `pmpcfg0` / `pmpaddr0`：物理内存保护相关寄存器，本次只实现基本读写。

5. 测试结果

最终运行：

```
make test-lab4
```

关键输出如下：

```
The first instruction of core 0 has committed. Diffptest enabled.  
Core 0: HIT GOOD TRAP at pc = 0x8001fff8  
total guest instructions = 32,766  
instrCnt = 32,766, cycleCnt = 158,411, IPC = 0.206842
```

这说明：

- Lab4 官方测试完整运行到 GOOD TRAP
- 6 条 CSR 指令的基本语义通过 Diffptest 检查
- CSR 状态与参考模型保持一致
- CSR 后的流水线刷新没有破坏程序执行
- 原有访存、分支和跳转路径在 Lab4 测试中仍能正常工作

6. 反思与总结

这次实验让我更清楚地认识到，CSR 虽然也是一种“寄存器”，但它和普通 GPR 不完全一样。

普通 GPR 可以依靠 forwarding 解决很多数据冒险，而 CSR 关系到处理器的全局状态。为了保证后续指令看到正确的 CSR 状态，CSR 指令后需要刷新流水线，而不是简单加入一条新的 forwarding 通路。

本次调试中最关键的问题是 Diffptest 的 CSR 状态晚一拍。这个问题和前面实验中的 GPR Diffptest 旁路很类似：提交当拍参考模型已经更新了状态，DUT 提供给 Diffptest 的状态也必须反映同一次提交后的结果。修复这个问题后，Lab4 测试能够稳定通过。

7. 大模型的使用说明

本次实验中，我使用 Cursor / 大模型工具作为辅助，主要用于：

- 梳理 Lab4 原始要求和 CSR 实现范围
- 分析现有五级流水线中适合接入 CSR 的位置

- 辅助定位 Difftest CSR mismatch 的原因
- 整理实验报告结构和表述

实际实现中的关键部分，包括 CSR 写入时机、CSR mask、流水线刷新策略和 Difftest CSR 连接，都结合代码、实验要求和测试结果进行了检查。